Chapter-One

INTRODUCTION TO C# AND .NET FRAMEWORK

Introduction to C# Language

- C# is a high-level general-purpose, Object-Oriented programming language which was developed by **Anders Hijlsberg.**
- It can be used to develop desktop application, web Application and Mobile Application.
- C# is a hybrid of C and C++ and it is a Microsoft programming language developed to compute with Sun's Java Programming language.
- C# is an object-oriented language used with XML based web services on the .Net platform and designed for improving productivity in the development of web application.
- C# is an elegant and type-safe object-oriented programming language that enables developers to build a variety of secure and robust applications that run on the .Net framework.
- It is also used to developed windows client applications, XML web services, distributed components, client-server applications, database applications and much more.
- All variables and methods including the Main method, the application's entry point are encapsulated within class definition.

Why use C#?

- It is modern general purpose programming language.
- It is object oriented.
- It is easy to learn.
- It is a structured language.
- It produces efficient programs
- It can be compiled on a variety of computer platforms.
- It is part of .Net Framework

Programming Languages used in .Net Framework

- Eleven programming languages which are designed and developed by Microsoft.
  I. C# .Net
  II. VB.Net
  III. C++ .Net
  IV. J# .Net

V. F#.Net
VI. JSCRIPT.Net
VII. Windows Power Sell
VIII. Iron Ruby
IX. Iron Python
X. C Omega
XI. ASML (Abstract State Machine Language)

IDE (Integrated Development Environment)

- Visual Studio is a very powerful Integrated Development Environment (IDE) where we actually write our C# or .Net programs.
- It is popular because it supports code editing interface design, server management, debugging, and performance analysis.
- We can download Visual Studio free.

Languages (WinForms, ADO.Net, ASP.Net)

- The types of applications that can be built in the .NET Framework is classified broadly into the following category:
  - WinForms
    It is used for developing forms-based applications which would run on an end user machine.
  - ADO.Net
    This technology is used to develop applications to interact with databases such as oracle or Microsoft SQL server (Active Data Object)
  - ASP.Net
    This technology is used for developing web based application, which are made to run on any browser such as softEdge, Chrome, Firefox, Opera etc.

Library (Framework Class Library (FCL))

- A class library is a collection of functions and methods that can be used for the core purpose.

Features of C#

1. **Object Orientation**

- ☞ C# is a rich implementation of the object orientation paradigm, which includes encapsulation, abstraction, inheritance and polymorphism.
- ☞ The distinctive features of C# from an object-oriented perspective are:
  - ❖ **Unified type system**
    Unified Type System (UTS) means that all data types are treated as objects and are derived from a single base class.

  - ❖ **Classes and interfaces**
  - ❖ **Properties, method and events**

☞ While C# is primarily an object-oriented language, it also borrows from the functional programming paradigm. Specifically:
- ❖ Function can be treated as value
- ❖ C# supports patterns for purity

2. Type Safety

☞ Type safety in C# refers to the language's ability to ensure that variables are of the correct type before they are used in an operation. This means that the compiler checks to make sure that the operations performed on variables are allowed based on their declared type.

☞ In C#, every variable has a specific type, which can be either a value type or a reference type. Value types store their values directly in memory, while reference types store a reference to a location in memory where the actual value is stored.

☞ Type safety in C# is enforced by the compiler at compile time, which means that type errors are caught before the program is run. This helps to prevent runtime errors that can occur when variables of the wrong type are used in an operation.

☞ For example, if you declare a variable as an integer and then try to assign a string value to it, the compiler will generate an error, because the operation is not allowed based on the declared type of the variable.

☞ C# is type safe object-oriented programming language unlike C language. It means that instances of types can interact only through protocols they define, thereby ensuring each type's internal consistency. For example,

int x="45"; in C language no error
In C# language it is an error.

```
int x = 5;
string y = "Hello";
int result = x + y; // Compile-time error: Cannot add int and string
```

Note:

C# type safety is a crucial feature that promotes code reliability and reduces the likelihood of runtime errors related to type mismatches. It encourages good coding practices by enforcing strict type rules, making it easier to write maintainable and bug-free code.

3. **Memory Management**

- C# relies on the runtime to perform automatic memory management. The Common language Runtime has a garbage collector that executes as part of your program, reclaiming memory for objects that are no longer referenced. This frees programmers from explicitly de-allocating the memory for an object, eliminating the problem of incorrect pointers encountered in language such as C++.
  - Stack (LIFO) Memory
    - Scope is local and gets allocated when method is invoked. It gets cleared when method returns.
  - Heap Memory
    - All dynamic requirements are fulfilled here. After the uses of the variable, it is returned to heap for further use.

4. **Platform Support**

- C# was used almost entirely for writing code to run on windows platforms. Recently, Microsoft and other companies have invested in other platforms, including Linux, mac OS, and Android.
- It allows cross platform C# development for mobile application and portable Class libraries are becoming increasingly widespread.
- Microsoft's ASP.NET Core is a cross-platform lightweight web hosting framework that can run either on the .NET framework or on .NET core an open source cross-platform runtime.

.NET Framework.

- Dot Net is software component developed by Microsoft which runs under operating system. .Net framework is platform which is released in the year 2002. Later on, several improvements have been taken place for the stronger and efficient platform.
- The .Net framework is a revolutionary platform that helps you to write the following types of applications:
  - **Windows applications**
    - A Windows application is a type of software program that runs on the Microsoft Windows operating system and provides a graphical user interface (GUI) for users to interact with.

- Unlike web applications, it is installed and executed directly on a local computer rather than accessed through a web browser.
- Windows applications are mainly developed using technologies like .NET Framework, C#, VB.NET, and tools such as Microsoft Visual Studio.
- They offer fast performance, rich user interfaces (buttons, forms, menus), and can work offline.

❖ **Web applications**

- A web application is a software program that users interact with through a web browser.
- It has a user interface (UI) (like buttons, forms, pages).
- Designed for human users.
- Uses technologies like HTML, CSS, JavaScript.

Example: Online banking system, e-commerce websites.

❖ **Web services**

- A web service is a software system that allows communication between applications over the internet.
- It does not have a user interface.
- Designed for machine-to-machine interaction.
- Uses protocols like HTTP, SOAP, REST, XML, JSON.
- Example: Payment gateway API, weather data API.

✍ The .Net framework applications are multi-platform applications. The framework has been designed in such a way that it can be used from any of the following languages: C#, C++, Visual Basic, Jscript, COBOL, etc.

✍ All these languages can access the framework as well as communicate with each other. .net is called .net framework because it supports multiple language, technology, and database.

✍ In dot net, we have nearly 60+ languages, only 9 languages were developed by Microsoft, other languages were developed by third party. In each we have multiple technologies such as ASP.NET, ADO.NET etc. and MS-Access, SQL server for database.

C# 1.0:
.NET Framework: .NET Framework 1.0/1.1
Visual Studio: Visual Studio .NET 2002

C# 2.0
.NET Framework: NET Framework 2.0
Visual Studio: Visual Studio 2005

C# 3.0
.NET Framework: .NET Framework 3.0\3.5
Visual Studio: Visual Studio 2008

C# 4.0
.NET Framework: .NET Framework 4.0
Visual Studio: Visual Studio 2010

C# 5.0
.NET Framework: .NET Framework 4.5
Visual Studio: Visual Studio 2012/2013

C# 6.0
.NET Framework: .NET Framework 4.6
Visual Studio: Visual Studio 2013/2015

C# 7.0
.NET Framework: .NET Core 2.0

Visual Studio: Visual Studio 2017

C# 8.0
.NET Framework: .NET Core 3.0
Visual Studio: Visual Studio 2019

C# 9.0
.NET Framework: .NET 5.0
Visual Studio: Visual Studio 2019

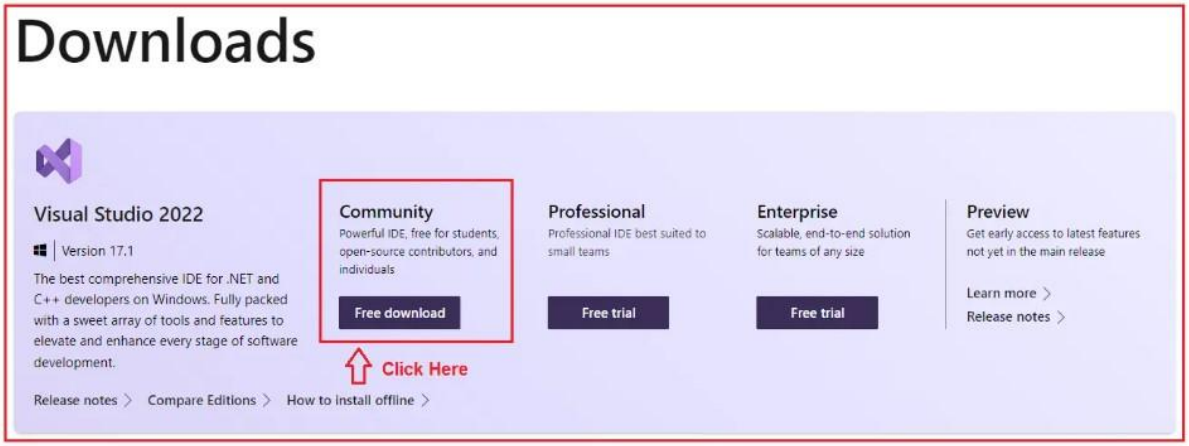
C# 10.0
.NET Framework: .NET 6.0
Visual Studio: Visual Studio 2022

C# 14.0
.NET Framework: .NET 10.0
Visual Studio: Visual Studio 2026

Download Visual Studio Latest Version

- ☞ In order to download visual studio, please visit the following Visual Studio download link

Download Visual Studio Tools - Install Free for Windows, Mac, Linux (microsoft.com)



Net Architecture:

- NET is tiered, modular, and hierarchal. Each tier of the .NET Framework is a layer of abstraction. .NET languages are the top tier and the most abstracted level.
- The common language runtime(CLR) is the bottom tier, the least abstracted, and closest to the native environment.
- This is important since the common language runtime works closely with the operating environment to manage .NET applications.
- The .NET Framework is partitioned into modules, each with its own distinct responsibility. Finally, since higher tiers request services only from the lower tiers, .NET is hierarchal. The architectural layout of the .NET Framework is illustrated in Figure below.

C#	**VC++**	**VB.NET**
Console and windows application	**Web application**	
ADO.NET and XML		
BCL (Base Class Library)/ FCL (Frame Class Library)		
CLR(Common Language Runtime)		
OS (operating system) (32 bit / 64 bit)		

Figure 1.1 : .Net Architecture

- NET Framework is a managed environment. The common language runtime monitors the execution of .NET applications and provides essential services. It manages memory, handles exceptions, ensures that applications are well-behaved, and much more.

- Language interoperability is one goal of .NET. .NET languages share a common runtime (the common language runtime, a common class library), the Framework Class Library (FCL), a common component model, and common types. In .NET, the programming language is a lifestyle choice. Except for subtle differences, C#, VB.NET, or JScript.NET offer a similar experience.

- .NET abstracts lower-level services, while retaining most of their flexibility. This is important to C-based programmers, who shudder at the limitations presented in Visual Basic 6 and earlier.

Component of .net framework

The main two components of .NET Framework are:

- Common Language Runtime (CLR)
- Base Class Library (BCL)/ Framework Class Library(FCL)

Common Language Runtime (CLR)

- The CLR is the execution engine for .NET applications and serves as the interface between .NET application and the operating system.
- It provides an environment to run all the .Net Programs. The code which runs under the CLR is called as **Managed Code**.
- Programmers need not to worry on managing the memory if the programs are running under the CLR as it provides memory management and thread management. The CLR provides many services such as:

  - Load and execute code.
  - Manages memory and object
  - Convert intermediate language into native code.
  - Exception handling.

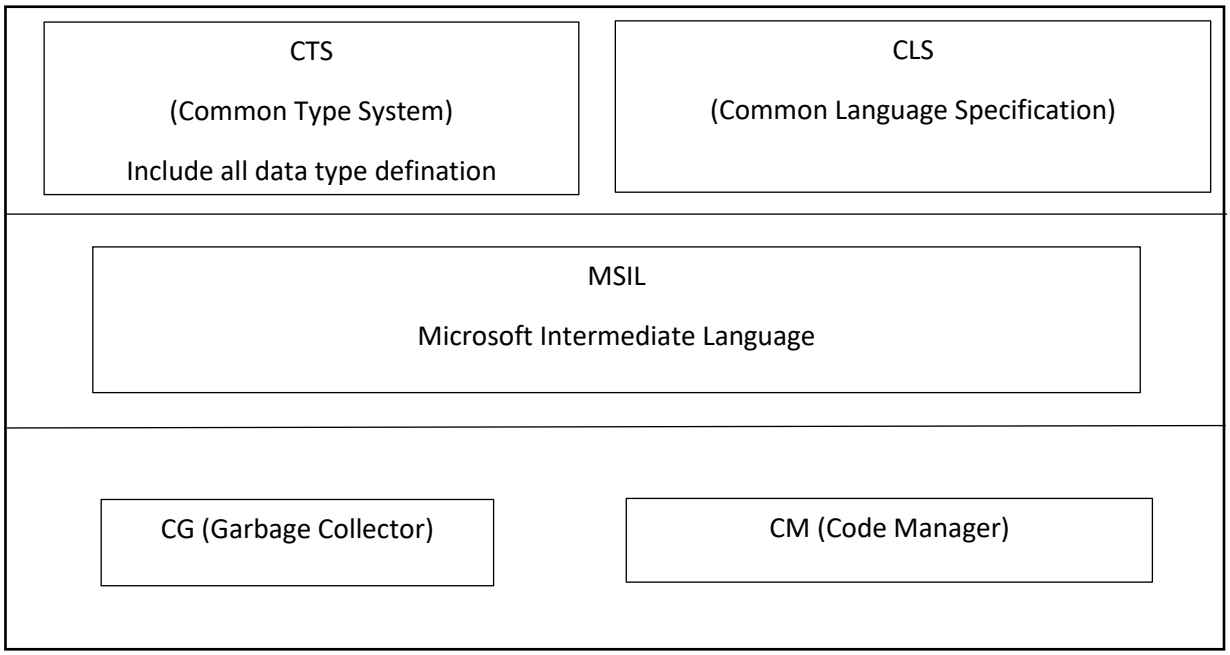


Fig: Architecture of CLR

Common Type System

- Common Type System (CTS) defines a common set of data types that all .NET languages use so that programs written in different languages can interact.
- CTS ensures type compatibility between languages, enabling cross-language integration in the .NET framework.
- There are two main types in CTS:
    - Value Types – stored in stack and passed by value.
    - Reference Types – stored in heap and passed by reference.
- Different languages have different data type names, but compilers convert them into common CTS types understood by the CLR.

Example:

- C# uses int
- VB.NET uses Integer

Common Language Specification

- Common Language Specification (CLS) is a set of basic rules that all .NET languages must follow to develop applications compatible with the .NET Framework.
- CLS ensures interoperability, meaning programs written in different .NET languages (like C#, VB.NET, VC++) can communicate and work together.
- Each programming language has its own syntax (e.g., C# statements end with ; while VB.NET does not), so languages cannot directly understand each other's syntax.

- The Common Language Runtime (CLR) understands all languages because they follow CLS rules.
- Code written in any .NET language is compiled into Intermediate Language (IL) by its compiler (e.g., C# → CSC compiler), and CLR executes this common IL code.

.Net Framework Class Library (FCL)

- .NET Framework Class Library (FCL) provides the core functionality of the .NET Framework and contains a large collection of reusable classes, interfaces, and value types.
- It simplifies and speeds up application development by giving programmers ready-made components to access system features.
- FCL is organized in a hierarchical structure and divided into Namespaces and Assemblies.

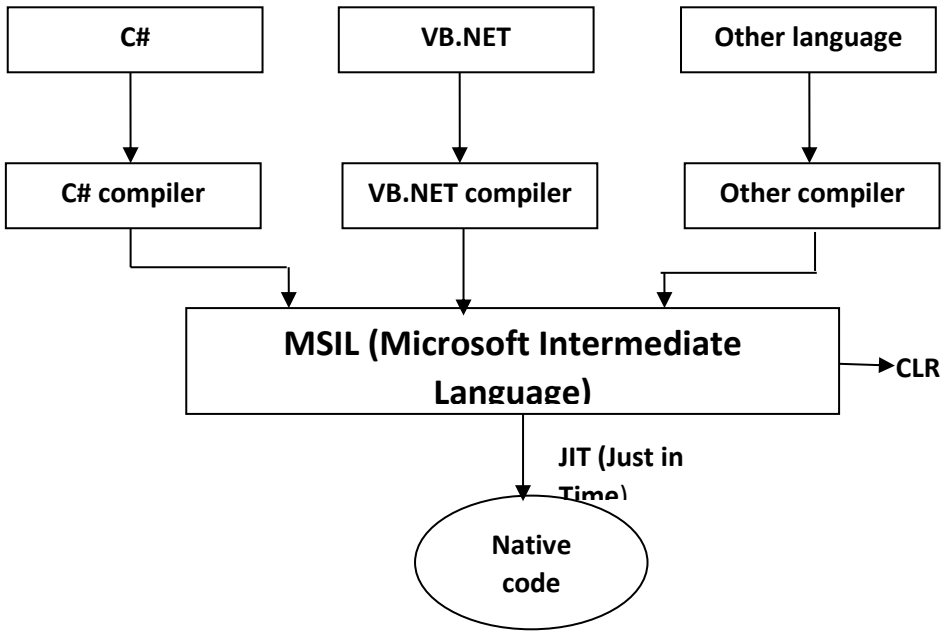
  - Namespaces: Logical grouping of related classes and types.
  - Assemblies: Compiled user-defined libraries.

- Assemblies can be of two types:

  - .dll (Dynamic Link Library) – used for code reusability, does not contain a main method.
  - .exe (Executable file) – contains the main method and is used to run the program.

- The System namespace is the root namespace of the .NET Framework.
- FCL classes are object-oriented, managed by CLR, and can also integrate with third-party components.

.Net Execution model



The execution of .net model consist of two phase.

- First program written in .net supporting language convert into Microsoft intermediate language (MSIL) using respective compiler.
- Second MSIL code can be converted through CLR into native code using JIT compiler.

MSIL (Microsoft Intermediate Language)

- MSIL (Microsoft Intermediate Language) is a language-independent code used in the .NET Framework.
- When a program is compiled in .NET, it is not directly converted into machine (native) code.
- Instead, the source code is compiled into MSIL code first.
- MSIL is independent of programming languages and operating systems, so code written in different .NET languages can work together.

- Later, the CLR converts MSIL into native machine code when the program runs.

JIT (Just-in-Time)

- JIT (Just-In-Time) Compiler converts MSIL code into native machine code.
- The generated native code is specific to the operating system and machine architecture.
- The operating system can execute the application only after JIT compilation.
- JIT compiles the code only when it is needed during program execution.
- This process improves performance and efficient execution of .NET applications.

JIT are of three types:

1. Pre JIT
2. Econo JIT
3. Normal JIT

Pre JIT

It converts all the code in executable code and it is slow

Econo JIT

It will convert the called executable code only. But it will convert code every time when a code is called again.

Normal JIT

It will only convert the called code and will store in cache so that it will not require converting code again. Normal JIT is fast.

.Net Standard 2.0

- .NET Standard is a specification that defines a set of APIs for Base Class Libraries (BCL).
- It helps unify different .NET platforms such as .NET Framework, .NET Core, Xamarin, Silverlight, and Xbox.
- It allows code sharing between applications running on different .NET platforms.
- Libraries developed using a specific .NET Standard version can work on any platform that supports that version.
- This improves compatibility and reusability of code across multiple .NET environments.
- .NET Standard is not a framework; it is only a specification.
- It defines a minimum set of APIs (types and members) that .NET platforms must support.
- It works similar to a C# interface, where frameworks implement the specification.
- It ensures compatibility between different .NET frameworks.
- It unifies Base Class Libraries (BCL) across .NET platforms.
- It enables sharing of libraries between different .NET platforms.
- It also helps centralize the evolution and development of BCL.

<table>
<tr><td>.NET FRAMEWORK</td><td>.NET CORE</td><td>XAMARIN</td></tr>
<tr><td>WPF
Windows Forms
ASP .NET</td><td>UWP
ASP.NET Core</td><td>iOS
Android
OS X</td></tr>
<tr><td colspan="3">.NET STANDARD LIBRARY

One library to rule them all</td></tr>
<tr><td colspan="3">COMMON INFRASTRUCTURE</td></tr>
<tr><td>Compilers</td><td>languages</td><td>Runtime Components</td></tr>
</table>

.NET Standard 2.0 is a new version that significantly increases the number of API's compared to the previous version (1.6.1). In fact, the API surface has more than doubled with .NET Standard 2.0

The .NET Standard 2.0 is supported by the following .NET implementations:

- .NET Core 2.0 or latter

- .NET Framework 4.6.1 or latter
- Mono 5.4 or later
- Xamarin.iOS 10.14 or later
- Xamarin.Mac 3.8 or later
- Xamarin.Android 8.0 or later
- Universal Windows Platform 10.0.16299 or later

What's new in the .NET Standard 2.0

- The .NET Standard 2.0 includes the following new features

  - A vastly expanded set of APIs
  - Support for .NET Framework libraries
  - Support for Visual Basic
  - Tooling support for .NET Standard libraries.

The CLR And Core FRAMEWORK

i. System Type

- The most fundamental types live directly in the System namespace.
- It includes C#'s built in types, the Exception base class, the Enum, Array, and Delegate base classes and Nullable, Type, Date Time, Timespan and Guid.
- It also includes type for performing mathematical functions (Math), generating random numbers (Random) and converting between various types (Convert and Bit Converter)

ii. Text Processing

- The System.Text namespace contains the stringBuilder class and the types for working with text encodings.
- The System.Text.RegularExpression namespace contains types that perform advanced pattern based search and replace operations.

iii. Collections

- The .Net Framework offers a variety of classes for managing collections of items. These incudes both list and dictionary based structure and work in conjunction with a set of standard interfaces that unify their common characteristics. All collection types are defined in the following namespaces.

```
System.Collections                //Nongeneric collection
System.Collection.Generic         //Generic collection
System.Collection.Specialized     //Strongly typed collections
System.Collection.ObjectModel     //Bases for your own collections
```

iv. Queries

- Language Integrated Query (LINQ) was added in Framework 3.5 LINQ allows you to perform type-safe queries over local and remote collections (e.g. SQL Server tables).
- A big advantage of LINQ is that it presents a consistent querying API across a variety of domains. The essential types reside in the following namespaces and are part of .Net Standard 2.0:

```
System. Linq                    //LINQ to Objects and PLINQ
System. Linq.Expressions        //for building experessions manullay
System.Xml. linq                // LINQ to XML
```

The full .Net Framework also includes the following,

```
System. Linq            //LINQ to SQL
System.Data.Entity      //linq to Entities (Entity Framework)
```

v. XML

- XMl used widely within the .NET Framework and so is supported extensively. The XMl namespaces are:

```
System.Xml                  //XmlReader, XmlWriter
System.Xml. linq            //The linq to XMl DOM
System.Xml.Schema           //Support for XSD
System.Xml.Sertialization //Declarative XMl serialization for .NET types
System.Xml.Xpath            //Xpath query language
System.Xml.Xsl              //Sylesheet support
```

vi. Streams and I/O

- The Framework provides a stream-based model for low-level input/output.
- The .NET Stream and I/O types are defined in and under the System.IO namespace and the WinRT types for file I/O are in and under Windows.Storage.

vii. Networking

- You can directly access standard network protocols such as HTTP, FTP, TCP/IP, and SMTP via the types in Stysem.Net.

```
System.Net
System.Net.Http             //Http Client
System.Net.Mail             //For sending mail via SMTP
System.Net.Sockets          //TCP, UDP and IP
```

viii. Serialization

- ☞ The framework provides several systems for saving and restoring objects to a binary or text representation. Such system are required for distributed application technologies such as WCF, Web Services and Remoting and also to save and restore objects to a file.
- ☞ The types for Serialization reside in the following namespace.

  System.Runtine.Serialization
  System.Xml.Serialization

ix. Security

- ☞ Code access, role and identity security and the transparency model introduced in CLR 4.0.
- ☞ Cryptography can be done in the Framework, covering encryption, hashing and data protection.
- ☞ The Type for this are as follows:
  System.Security
  System.Security.Permissions
  System.Security.Policy
  System.Security.Cryptography

APPLIED TECHNOLOGIES

- ☞ .NET Implementation and their descriptions are listed in table below:

	Operating System	Open Source	Purpose
.NET Framework	Windows	No	Uses for building Windows desktop application and ASP.NET web apps running in IIS server.
.NET Core	Windows, Linux, macOS	Yes	Used for building cross-platform console apps and ASP.Net core Web apps and cloud services.
Xamarin	ios, Android,macOS	Yes	Use for building mobile applications for iOS and Android as well as desktop apps for macOS.
.NET Standard	N/A	Yes	Used for building libraries that can be referenced from all .Net implementations such as .NET Framework .Net Core and Xamarin.

Some important applied technologies are listed below:

i. User Interface APIs
ii. ASP .NET
iii. ASP .NET Core

iv. Windows Presentation Foundation (WPF)
v. Windows Form
vi. Xamarin
vii. Universal Windows Platform (UWP)
viii. Silverlight

i. User Interface APIs

- User Interface based applications can be divided into two categories: thin client, which amounts to a website and rich client, which is a program the end user must download and install on a computer or mobile device. For thin client applications, .NET provides ASP.NET and ASP.NET Core.
- For rich-client applications that target windows 7/8/10/11 desktop, .NET provides the WPF and windows Forms APIs. It targets iOS, Android and Windows Phone, there's Xamarin and for writing rich-client store apps for Windows 10 desktop and devices.
- Finally, there's a hybrid technology called Silverlight, which has been largely abandoned since the rise of HTML5.

ii. ASP .NET

- Applications written using ASP.NET host under Windows IIS Server and can be accessed from any web browser.
- The advantages of ASP.NET over rich-client technologies are:
  - There are zero deployment at the client end.
  - Clients can run a non-windows platform.
  - Updates are easily deployed.

iii. ASP .NET Core

- A relatively recent addition, ASP.NET Core is similar to ASP .NET, but runs on both .NET Framework and .NET Core (allowing for cross-platform deployment).
- ASP .NET Core features a light-weight modular architecture, with the ability to self-host in a custom process and an open source license.

iv. Windows Presentation Foundation(WPF)

- WPF was introduced in Framework 3.0 for writing rich-client applications. The benefits of WPF over its predecessors, Windows Forms are as follows:
  - It supports sophisticated graphics, such as arbitrary transformations, 3D rendering templates.
  - Its primary measurement unit is not pixel-based, so applications display correctly at any DPI (dot per inch) setting.
  - It has extensive and flexible layout support, which means you can localize an application without danger of elements overlapping.
  - Rendering uses DirectX and is fast, taking good advantage of graphics hard-ware acceleration.
  - It offers reliable data binding.

- ❖ User interface can be described declaratively in XAMl files that can be maintained independently of the "code-behind" files this helps to separate appearance from functionality.

v. Windows Forms

- ☞ Windows Forms is a rich-client API that's as old as .NET Framework. Compared to WPF, Windows Forms is a relatively simple technology that provides most of the features you need in writing a typical Windows application. It also has significant relevancy in maintaining legacy applications.

vi. Xamarin

- ☞ Xamarin, now owned by Microsoft, lets you write mobile apps in C# that target iOS and Android, as well as windows Phone. Being cross-platform, this runs not on the .NET Framework, but its own framework (a derivation of the open source Mono framework).

vii. Universal Windows Platform

- ☞ UWP is for writing apps that target windows-10 desktop and devices, distributed via the Windows Store.
- ☞ Its rich-client API is designed for writing touch-first user interfaces and was inspired by WPF and uses XAML for layout. The namespaces are Windows UI and Windows.UI.Xaml.

viii. Silverlight

- ☞ Silverlight is also distinct from the .NET Framework and lets you write a graphical UI that runs in a web browser much like Macromedia's Flash with the rise of HTML5, Microsoft has abandoned Silverlight.

BACKEND TECHNOLOGY

- ☞ Backend Technology in the .NET Framework refers to the server-side technologies used to build the logic, database interaction, and processing of web or desktop applications. It handles tasks such as processing user requests, accessing databases, authentication, and generating responses.
- ☞ The following are the different backend technologies associated with .NET Framework:

1. ADO.NET

- ☞ ADO.NET is used to connect .NET applications with databases. It allows applications to retrieve, insert, update, and delete data from databases such as Microsoft SQL Server.
- ☞ ADO.NET is the managed data access API. Although the name is derived from the ADO (ActiveX Data Objects), the technology is completely different. ADO.NET contains two major low-level components:

a. Provider layer

- Provider model defines common classes and interfaces for low-level access to database providers.
- These interfaces comprise connections, commands, adapters and readers (forward-only, read-only cursors over a database).
- The Framework ships with native support for Microsoft SQL Server and numerous third-party drivers are available for other databases.

b. DataSet model

- A data set is a structure cache of data.
- It resembles a primitive in-memory database, which defines SQL, construct such as tables, rows, columns, relationships, constraints, and views.
- By programming against a cache of data, you can reduce the number of trips to the server, increasing server scalability and the responsiveness of a rich-client user interface.
- Data Set are serializable and are designed to be sent across the wire between client and server application.
- Sitting above the provider layer are three API's that offer the ability to query databases via LINQ :
    - Entry Framework(.Net Framework Only)
    - Entry Framework Core(.NET Framework and .NET Core)
    - LINQ to SQL (.NET Framework only)

2. Windows Workflow(.NET Framework Only)

- Widows workflow is a Framework for modelling and managing potentially long running business processes.
- It targets a standard runtime library, providing consistency and interoperability.
- It also helps to reduce coding for dynamically controlled decision making trees.
- Windows workflow is not strictly a backend technology, you can use it anywhere.
- Workflow came originally with .NET Framework 3.0, with its types defined in the **System.WorkFlow** namespace.
- Workflow was substantially revised in Framework 4.0 the new types live in and under the **System.Activities** namespace.

3. COM + and MSMQ (.NET Framework only)

- COM+ is a component-based technology in the .NET Framework used to build distributed and scalable applications. It extends the features of Component Object Model (COM) by providing additional services for enterprise applications.

- ☞ The framework allows you to interoperate with COM+ for services such as distributed transactions, via types in the **System.EnterpriseServices** namespace.
- ☞ Some important features of COM+:
    - **Transaction Management:** Ensures that multiple operations are completed successfully as a single unit.
    - **Object Pooling:** Reuses objects to improve application performance.
    - **Role-Based Security:** Controls access to application components based on user roles.
    - **Automatic Activation:** Components are activated only when needed.
    - **Distributed Application Support:** Allows components to communicate across different machines.
- ☞ Microsoft Message Queuing (MSMQ) is a message-based communication technology in the .NET Framework. It enables applications running at different times or on different machines to communicate with each other through message queues.
- ☞ It also supports MSMQ (Microsoft Message Quering) for asynchronous, one-way messaging through types in System.Messaging.
- ☞ Some important features of MSMQ are:
    - **Asynchronous Communication:** Applications send messages to a queue without waiting for the receiver.
    - **Reliable Message Delivery:** Messages are stored in queues until the receiver processes them.
    - **Message Prioritization:** Messages can be assigned different priority levels.
    - **Offline Communication:** Messages can be delivered even if the receiver is temporarily offline.
    - **Security:** Supports encryption and authentication for message transmission.

DISTRIBUTED SYSTEM TECHNOLOGIES

- ☞ A distributed system technology is computing environment in which various components are spread across multiple computer or devices on a network.
- ☞ These devices split up the work, coordination their efforts to complete the job more efficiently than if a single device had been responsible for the task.
- ☞ The different distributed technologies are:

1. Windows Communication Foundation (WCF)
2. Web API
3. Remoting and .ASMX web services (.NET Framework only)

1. **Windows Communication Foundation (WCF)**
   - ☞ Windows Communication Foundation is a sophisticated communications infrastructure introduced in Framework 3.0, WCF is flexible and configurable enough to make both of its predecessors- Remoting and (.ASMX) Web Services- mostly redundant.
   - ☞ The types for communicating with WCF are in and below the **System.ServiceModel** namespace.
   - ☞ WCF, Remoting, and Web Services are all alike in that they implement the following basic model in allowing a client and server application to communicate:
     - ❖ On the server, you indicate what methods you would like remote client applications to be able to call.
     - ❖ On the client, you specify or inter the signatures of the server methods you would like to call.
     - ❖ On both the server and the client you choose a transport and communication protocol (in WCF, this is done through a binding).
     - ❖ The client establishes a connection to the server.
     - ❖ The client calls a remote method, which executes transparently on the server.
2. **Web API**
   - ☞ Web API runs over ASP.NET/ASP.NET Core and is architecturally similar to Microsoft's MVC API, accept that it's designed to expose services and data instead of web page.
   - ☞ It's advantage over WCF is in allowing you to follow popular REST over HTTP conventions, offering easy interoperability with the widest range of platforms.
   - ☞ REST implementations are internally simpler than the SOAP and WS-protocols that WCF relies on for interoperability.
   - ☞ REST API's are also architecturally more elegant for loosely-coupled systems, building on de-facto standards and making excellent use of what HTTP already provides.

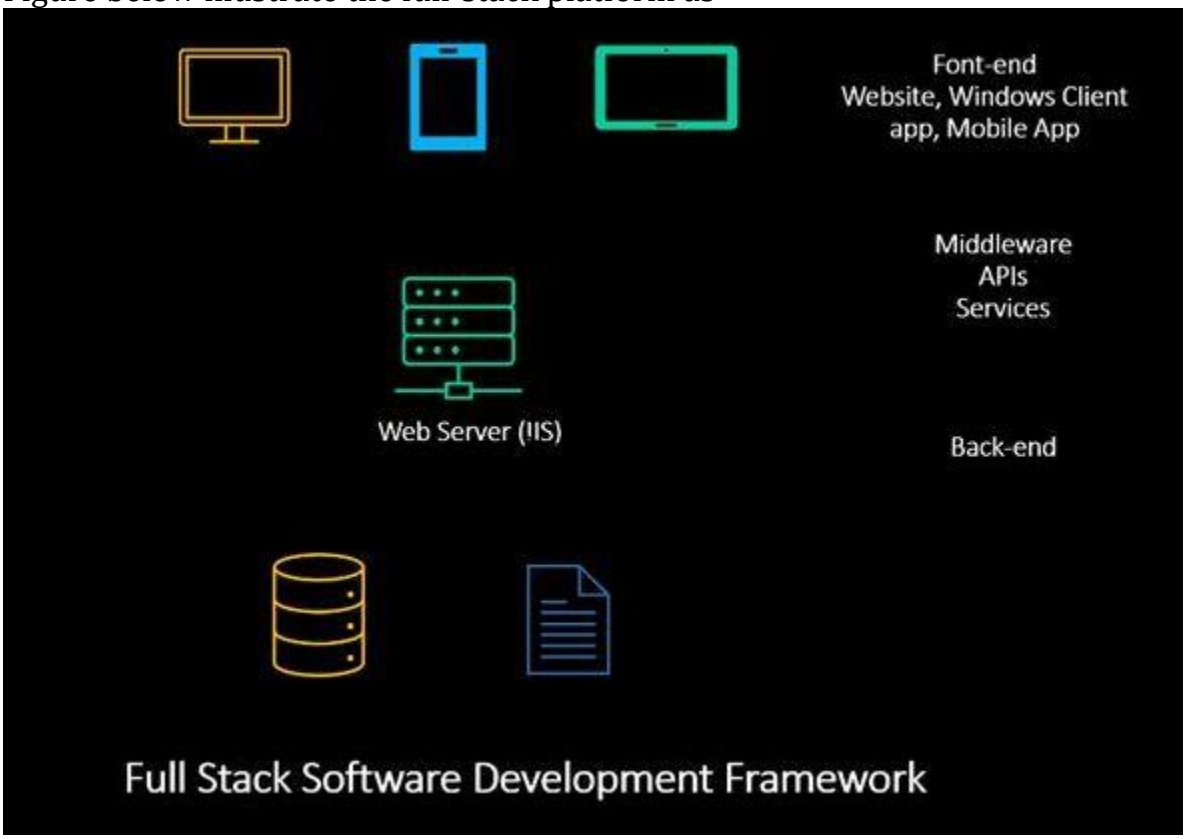
3. **Remoting and .ASMX web services (.NET Framework only)**
   - ☞ Remoting and .ASMX web services are WCF's predecessors. Remoting is almost redundant in WCF's wake and .ASMX Web Services has become entirely redundant.
   - ☞ Remoting is geared toward tightly coupled applications.
   - ☞ The types for Remoting are in or under **System.Runtime.Remoting**, the types for web services are under **System.Web.Servies.**

SCOPE OF .NET TECHNOLOGY

- Some important scope of .NET Technologies are as follows:

1. .NET is a full-stack platform

- A typical software application has three layers – front-end, middleware, and back-end. In the case of a web application, the front-end is the web pages that are usually written in HTML and JavaScript hosted in ASP.NET.
- A middleware is usually written in C# as a library or API. A back-end is a code that runs on a Web server (IIS) and talks to a data source.
- If you're a front-end developer in the .NET ecosystem, you will most likely work with ASP.NET Core with a combination of HTML 5, CSS, JavaScript, and other JavaScript libraries.
- If you're a back-end developer, you will most likely to build libraries, APIs, business logic, and data access code using C# language. The point here is, unlike several other frameworks, .NET 5 is a full-stack framework that can be used to build entire software applications from front-end to back-end.
- Figure below illustrate the full-stack platform as



2. .NET is unified

- Today, software runs the world. Software is not just for web, windows, or mobile. Your car is managed by a software. Your TV runs on software. NET. New homes are powered by software that includes thermostats,

bulbs, fans, security, internet, and door locks. Weather is monitored by software. Your watch notifications are a work of software. .NET is a unified platform to build app kinds of software applications.

- ☞ .NET 5 can be used to build desktop, web, cloud, mobile, gaming, IoT, and AI applications.

3. **.NET is open source**
   - ☞ .NET and C# projects are open source and are managed by Microsoft and the open-source community. They are available on Github. .NET 5.0 supports both Windows and Linux operating systems.

4. **.NET has a large community support**
   - ☞ .NET community is one of the largest communities, with millions of software developers involved in contributions around the globe. C# Corner alone has millions of developers visit each month to learn and share to the .NET ecosystem.
5. **.NET is friendly**
   - ☞ .NET 5.0 is friendly with other frameworks and libraries. For example, if you're building a .NET web application, you can easily use other popular open-source libraries and frameworks as a part of your .NET application. You can build a large web application using ASP.NET Core that can integrate with Angular, TypeScript, and JavaScript.

FEATURES OF OBJECT-ORIENTED PROGRAMMING

- ☞ The Object-Oriented Programming refers to a type of computer programming in which programmers define not only the data type of a data structure, but also the types of operation i.e. method or function that can be applied to the data structure.

- Some important features of object-Oriented Programming are as follows:

i. Object

- A self-contained entity that consists of both data and function to manipulate the data.

ii. Class

- A group of object is known as class. The class defines all the common properties of the different objects that belong to it.

iii. Data Abstraction

- The process of picking out (abstracting) common features of objects and procedures.

iv. Encapsulation

- The process of combining elements to create a new entity. A procedure is a type of encapsulation because it combines a series of computer instructions.

v. Polymorphism

- Polymorphism means one name multiple form.
- A programming languages ability to process objects differently depending on their data type or class.

vi. Inheritance

- A feature that represents the "is a" relationship between different classes.

vii. Interface

- The languages and codes that the applications use to communicate with each other and with the hardware.

viii. Information Hiding

- The process of hiding details of an object or function.
- Information hiding is a powerful programming technique because it reduces complexity.

ix. Procedure

- A section of a program that performs a specific task.

Difference between Object Oriented Programming and Procedural Oriented Programming

Procedure Oriented Programming(POP)	**Object Oriented Programming (OOP)**
In POP, program is divided into small parts called **function or method**.	In OOP, program is divided into parts called **Object**.
In POP, Importance is not given to data but to functions as well as sequence of actions to be done.	In OOP, Importance is given to the data rather than procedures or function because it works as a real world.
POP follows Top Down Approach.	OOP follows Bottom Up approach
POP does not have any access specifier.	OOP has access specifiers named Public, Private, Protected etc.
In POP, Data can move freely from function to function in the system.	In OOP, objects can move and communicate with each other through member function.
To add new data and function in POP is not so easy.	OOP provides an easy way to add new data and function.
In POP, Most function uses Global data for sharing that can be accessed freely from function to function in the system.	In OOP, data cannot move easily from function to function, it can be kept public or private so we can control the access of data.
POP does not have any proper way for hiding data so it is less secure.	OOP provides Data Hiding so provides more security.
In POP, Overloading is not possible	In OOP, overloading is possible in the form of Function Overloading and Operator Overloading.
Example of POP programming are: C, VB, FORTRAN, Pascal.	Example of OOP are: C++, JAVA, VB.NET, C# with .NET.